1 Brute Force Classification

1.1 Supervised Machine Learning

- Input space X, Output space Y
- Training data: given many instances $\mathcal{D} = \{(\vec{x}_i, y_i)\}_{i=1}^{N}$
- Learn a function $X \to Y$ which is expressed by parameters
- Test data: Given new $\vec{x}_{N+1}$, use function to predict y_{N+1}
- Evaluation: Check whether prediction y_{N+1} matches the ground truth y^*_{N+1}
- y^*_{N+1} is not known to the black box (held-out)

1.2 Text Categorization

- Given a text document, assign it to one of K categories.
- Consider how you're deciding the category and how you would explain it to the computer.
- Input space: documents, Output: one of K categories
- Training data: documents and their category (e.g. sports, news)
- Learn a categorization function
- Test data: Given a new document, use function to predict category
- Evaluation: Check whether predicted category is correct
- Evaluation measure: how many correct predictions?

1.2.1 Binary Text Categorization Task

- There is only one category.
- Predict whether the document is in the category or not (pos/neg)
- Same: Predict the set of documents that are in this category.
- Input space: documents, Output: $\{0, 1\}$ (1=positive)
- Training data: documents and whether they are in this category (e.g. sports or not)
- Learn a categorization function
- Evaluation measure: how many correct predictions?

1.2.2 One-vs-Rest Classification

- Input space: documents, Output: one of K categories
- Redefine the task to learn K binary categorization functions
- Learn K categorization functions (one per category)
- Can a document be positive for multiple categories? If yes, which category to choose?
  - Each categorization function bases the decision on a **Classification Score** or **Confidence**
  - Assign the document to the category with the highest confidence

1.2.3 Supervised Document Classification

- Represent documents with feature vectors
- Example content features:
  - $f_1 d$: 1 if doc d contains the word "ball"; 0 otherwise
  - $f_2(d)$: 1 if doc d contains the world "goal"; 0 otherwise
  - $f_3(d)$: 1 if doc d contains the word "election"; 0 otherwise
- Change the parameters in the function to get more predictions right
- Iteratively update parameters (Training loop)
- Then fix the parameters, make one prediction in the test set
- Sometimes: very good on the train set, but errors on the test set
- **Note: Never train on the test data!**
  - Never test on the training data (unless you call it training error)
  - Never leak the test data into your training process

1.3 A Simple Categorization Model

1.3.1 Linear Hyperplane Model

- Feature vectors in space X (number of features = dimension of space)
- Find a hyperplane that separates positive from negative examples
- To classify a new example, see if it is on the "positive" side of the hyperplane
  - Weight vector $\vec{w}$ points towards positive side
- *The hyperplane has one fewer dimension than the vector space*

1.3.2 Hyperplane Parameters

- Hyperplane is represented by vector $\vec{w}$
- $\vec{w}$ is orthogonal to the hyperplane
- $\vec{w}$ has the same dimensionality as $\vec{x}_i$
- If $\vec{w} \cdot \vec{x}_i > 0$ then example i is positive
- Inner product $\vec{w} \cdot \vec{x}_i = \sum_j w_j x_{ij}$ over feature dimensions j.

1.3.3 Inner Products and Cosines

- If $\vec{w} \cdot \vec{x} > 0$ then example i is positive
- Same as: $\angle(\vec{w}, \vec{x}_i) < 90\,\text{deg}$
- Same as: $\cos(\vec{w}, \vec{x}_i) > 0$ $\qquad$ $\cos(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{||\vec{a}||\,||\vec{b}||}$

1.3.4 Hard vs Soft Classification

- Sometimes we want to tolerate classification errors in training data to get a more robust hyperplane
- We don't want outliers and errors to dictate our model.
- **Hard classification** is strict; the hyperplane exactly and entirely divides the space into positive and negative
- **Soft classification** allows for some errors, typically to allow for a simpler hyperplane. This also prevents outliers and errors from causing issues with classification

1.4 The Intercept of the Model

1.4.1 Changing the Threshold with "The Intercept"

- Changing the threshold $b : \vec{w} \cdot \vec{x}_1 > b$ $\qquad$ The intercept b